

PHẦN 1: TÌM 3 API PUBLIC THẬT VỀ MỘT DOMAIN QUAN TÂM

1. Jikan API (Lĩnh vực Anime/Manga)

- Loại API: REST API
- Base URL: <https://api.jikan.moe/v4/>
- Auth method: Không yêu cầu API key đối với các endpoint public thông thường.
- Có versioning không?: Có. Version được thể hiện trực tiếp trong URL thông qua /v4/.
- Resource identifier: Các resource như anime, manga, character được định danh bằng các ID số nguyên, thường là ID tương ứng trên MyAnimeList.
- Dữ liệu cung cấp: Jikan cung cấp dữ liệu về anime, manga, nhân vật và nhiều thông tin liên quan từ MyAnimeList.

2. GitHub REST API (Lĩnh vực Phát triển phần mềm)

- Loại API: REST API
- Base URL: <https://api.github.com>
- Auth method: Personal Access Token hoặc OAuth. Token được gửi qua HTTP Header:
Authorization: Bearer {token}
- Có versioning không?: Có. GitHub sử dụng versioning theo ngày thông qua HTTP Header, ví dụ:
X-GitHub-API-Version: 2022-11-28
- Resource identifier: GitHub sử dụng các ID số cho nhiều resource như user, issue, pull request; đối với commit có thể sử dụng mã SHA để định danh.

3. Football-Data API (Lĩnh vực Thể thao - Bóng đá)

- Loại API: REST API
- Base URL: <https://api.football-data.org/v4/>
- Auth method: API Key. API key được truyền thông qua Header:
X-Auth-Token: {api_key}
- Có versioning không?: Có. Version được thể hiện trực tiếp trong URL, ví dụ /v4/.
- Resource identifier: Các resource như competition và team được định danh bằng các ID dạng số nguyên.

PHẦN 2: ĐỌC HIGGINBOTHAM CH.1 + GEEWAX CH.1 VÀ SO SÁNH

1. Định nghĩa API

API là cơ chế cho phép 2 thành phần phần mềm giao tiếp với nhau bằng một tập hợp các định nghĩa và giao thức. Ví dụ: hệ thống phần mềm của cơ quan thời tiết chứa dữ liệu về thời tiết hàng ngày. Ứng dụng thời tiết trên điện thoại của bạn sẽ “trò chuyện” với hệ thống này qua API và hiển thị thông tin cập nhật về thời tiết hàng ngày trên điện thoại của bạn.

2. Định nghĩa Resource

Resource là một object hoặc đại diện cho đối tượng và nó có một tập hợp các methods liên quan đến nó. Ví dụ: employee, user, ... là các resources và delete, add, update là các method để thao tác với các resources.

* 5 đặc điểm cốt lõi của một API tốt:

1. Có hợp đồng rõ ràng (Clear Contract)

Endpoint, method, request body, response shape, error codes được mô tả trước khi viết code. Client có thể đọc spec mà không cần đọc code.

2. Che giấu chi tiết cài đặt (Abstraction)

Client không biết, và không nên biết, server xử lý thế nào. Database là gì, ngôn ngữ gì, infra ra sao đều là chi tiết triển khai nội bộ

3. Có tính tái sử dụng (Reusability)

Một client (có thể từ team khác) gọi được nhiều lần mà không cần code phía server phải đổi. API dùng để chia sẻ chức năng chứ không copy/paste.

4. Độc lập với ngôn ngữ và nền tảng (Language-agnostic)

Một API HTTP/JSON có thể được gọi từ Python, Node.js, Go, Java, PHP, .NET, mobile. Định dạng truyền thống JSON, XML, Protobuf là trung gian, không ai "sở hữu" API.

5. Có khả năng phát triển và quản lý (Versioning & Observability)

Một API chuyên nghiệp phải có version (v1, v2...) để tiến hóa mà không phá client cũ, kèm logs, metrics, traces để vận hành (nội dung buổi 9 và 10)

*So sánh với 5 đặc điểm đã học trong Slide 05 — Higginbotham và Geewax có bổ sung gì?

1. Higginbotham bổ sung góc nhìn “API như một sản phẩm” (API as a Product)

So với Slide 05 chủ yếu tập trung vào đặc điểm kỹ thuật của API, Higginbotham nhấn mạnh rằng API cần được xem như một sản phẩm phục vụ developer.

API không chỉ cần hoạt động đúng mà còn phải dễ hiểu, dễ sử dụng và mang lại trải nghiệm tốt cho người sử dụng API. Từ đó, ông đề cao cách tiếp cận Outside-In: bắt đầu từ nhu cầu của client/developer rồi mới thiết kế API, thay vì xây dựng API chỉ dựa trên cấu trúc database hoặc cách hệ thống bên trong được triển khai.

→ Đây là điểm bổ sung quan trọng về góc nhìn thiết kế và trải nghiệm người dùng API.

2. Geewax bổ sung nguyên tắc Consistency (Tính nhất quán)

Các đặc điểm trong Slide 05 đề cập đến hợp đồng và khả năng sử dụng API, nhưng Geewax nhấn mạnh sâu hơn vào tính nhất quán trong toàn bộ API.

Ví dụ, các endpoint nên có cách đặt tên thống nhất, các field nên được đặt tên theo cùng một quy tắc, response có cấu trúc nhất quán và cách trả về lỗi cũng nên được thiết kế thống nhất.

→ Tính nhất quán giúp developer dễ học API hơn và có thể dự đoán cách hoạt động của một endpoint mới dựa trên những endpoint đã biết.

PHẦN 3: HOÀN THIỆN BÀI 6 + MỞ RỘNG

1. Yêu cầu

- CRUD Books
- Tìm kiếm bằng q
- Sort theo title
- year $\geq$ 1900

2. Thiết kế API

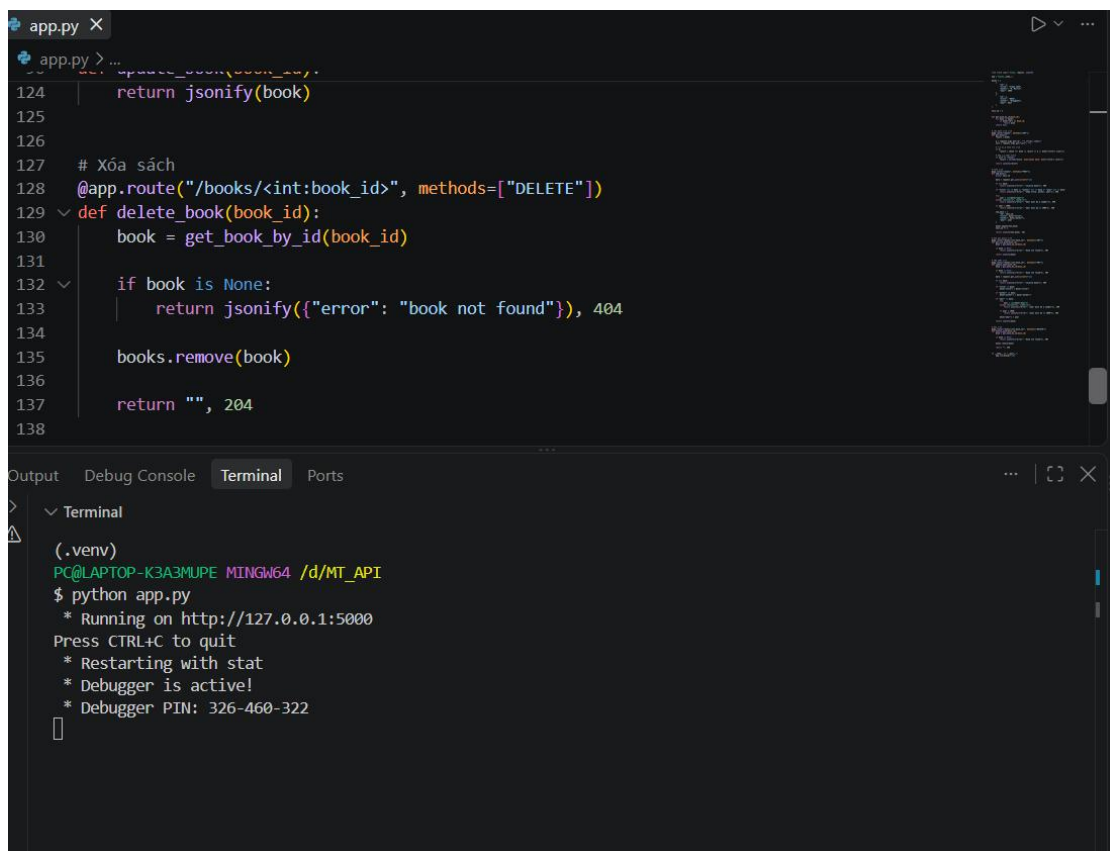
- GET /books
- POST /books
- GET /books/<id>
- PUT /books/<id>
- DELETE /books/<id>

3. Mô tả chức năng

- Tìm kiếm
- Sắp xếp
- Validate year
- Xử lý lỗi 404, 400

4. Kết quả kiểm thử

- Ảnh GET
- Ảnh POST
- Ảnh PUT
- Ảnh DELETE
- Ảnh trường hợp year < 1900



```
app.py X
app.py > ...
124 |     return jsonify(book)
125 |
126 |
127 | # Xóa sách
128 | @app.route("/books/<int:book_id>", methods=["DELETE"])
129 | def delete_book(book_id):
130 |     book = get_book_by_id(book_id)
131 |
132 |     if book is None:
133 |         return jsonify({"error": "book not found"}), 404
134 |
135 |     books.remove(book)
136 |
137 |     return "", 204
138 |

Output Debug Console Terminal Ports
Terminal
(.venv)
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ python app.py
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 326-460-322
```

```
app.py X
app.py > ...
1  from flask import Flask, request, jsonify
2
3  app = Flask(__name__)
4
5  books = [
6      {
7          "id": 1,
8          "title": "Clean Code",
9          "author": "R. Martin",
10         "year": 2008
11     },
12     {
13         "id": 2,
14         "title": "DDIA",
15         "author": "Kleppmann",
16         "year": 2017
17     }
18 ]
19
20 if __name__ == '__main__':
21     app.run(debug=True)
```

Output Debug Console Terminal Ports

Terminal

```
$ source /d/MT_API/.venv/Scripts/activate
(.venv)
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a product
ead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 326-460-322
127.0.0.1 - - [14/Sep/2026 13:30:41] "POST /books HTTP/1.1" 201 -
127.0.0.1 - - [14/Sep/2026 13:31:09] "POST /books HTTP/1.1" 400 -
127.0.0.1 - - [14/Sep/2026 13:31:22] "GET /books?q=clean HTTP/1.1" 200 -
127.0.0.1 - - [14/Sep/2026 13:31:30] "GET /books?sort=title HTTP/1.1" 200 -
```

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl -i -X POST http://127.0.0.1:5000/books -H "Content-Type: application/json" -d '{"title": "DDIA", "author": "Kleppmann", "year": 2017}'
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 14 Sep 2026 06:30:41 GMT
Content-Type: application/json
Content-Length: 74
Connection: close

{
  "author": "Kleppmann",
  "id": 3,
  "title": "DDIA",
  "year": 2017
}
(.venv)
```

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl -i -X POST http://127.0.0.1:5000/books -H "Content-Type: application/json" -d '{"title": "Old Book", "author": "Author", "year": 1850}'
HTTP/1.1 400 BAD REQUEST
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 14 Sep 2026 06:31:09 GMT
Content-Type: application/json
Content-Length: 38
Connection: close

{
  "error": "year must be >= 1900"
}
(.venv)
```

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl -i "http://127.0.0.1:5000/books?q=clean"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 14 Sep 2026 06:31:22 GMT
Content-Type: application/json
Content-Length: 96
Connection: close

[
  {
    "author": "R. Martin",
    "id": 1,
    "title": "Clean Code",
    "year": 2008
  }
]
(.venv)
```

```
PC@LAPTOP-K3A3MUPE MINGW64 /d/MT_API
$ curl -i "http://127.0.0.1:5000/books?sort=title"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.14.3
Date: Mon, 14 Sep 2026 06:31:30 GMT
Content-Type: application/json
Content-Length: 270
Connection: close
```

```
[
  {
    "author": "R. Martin",
    "id": 1,
    "title": "Clean Code",
    "year": 2008
  },
  {
    "author": "Kleppmann",
    "id": 2,
    "title": "DDIA",
    "year": 2017
  },
  {
    "author": "Kleppmann",
    "id": 3,
    "title": "DDIA",
    "year": 2017
  }
]
```